AI-Lingo

State Diagram & Class Diagram

System Analysis & Design – Data Flow & System Behavior

---

4. State Diagram

4.1 Purpose and Scope

A state diagram shows the states an object passes through during its life and the events that cause it to move from one state to another. It complements the Sequence and Activity diagrams: those describe how components interact and how a process flows, while the state diagram describes how the status of a single object changes over time.

Four objects with a meaningful life cycle were selected. Each one has a status field in the Database Design (Firestore and Hive structures) and is driven by the use cases defined in the System Analysis section.

Object	Status values (source)	Use cases	Functional requirements
Saved word (UserWord)	new, learning, learned (UserWord.status)	UC-06	FR-08, FR-09
Lesson	generated, in_progress, completed (Lesson.status)	UC-04, UC-05	FR-05, FR-06, FR-07
Level assessment	in_progress, completed (LevelAssessment.status)	UC-02	FR-03
Sync operation	pending, syncing (life cycle of a SyncOperation in Hive syncQueueBox)	UC-09	FR-10, FR-11, FR-12

Notation and colour key

Element	Meaning
Filled black circle	Initial state: where the life of the object begins.
Circle with an outer ring	Final state: the object is completed or removed.
Rounded rectangle	A state. The lower compartment lists entry actions executed when the state is entered.
Arrow label	event [guard] / action – the event that triggers the transition, an optional condition, and the action performed.
Dashed outline region	A composite state: a group of states that share a transition (for example, deleting a saved word from any of its states).
Colours	Blue: start state and normal transition. Orange: active state and repeated (self) transition. Green: successful completion. Red arrow: regression, failure or removal. Purple dashed arrow: alternative entry.

4.2 State Diagram – Saved Word

A word becomes a UserWord when the learner saves it (UC-06). It starts as new, becomes learning after the first review, and is marked learned once the learner has answered correctly N times. An incorrect answer sends a learned word back to learning. Deleting the word, or the whole account, can happen from any of the three states, so it is drawn from the enclosing composite state.

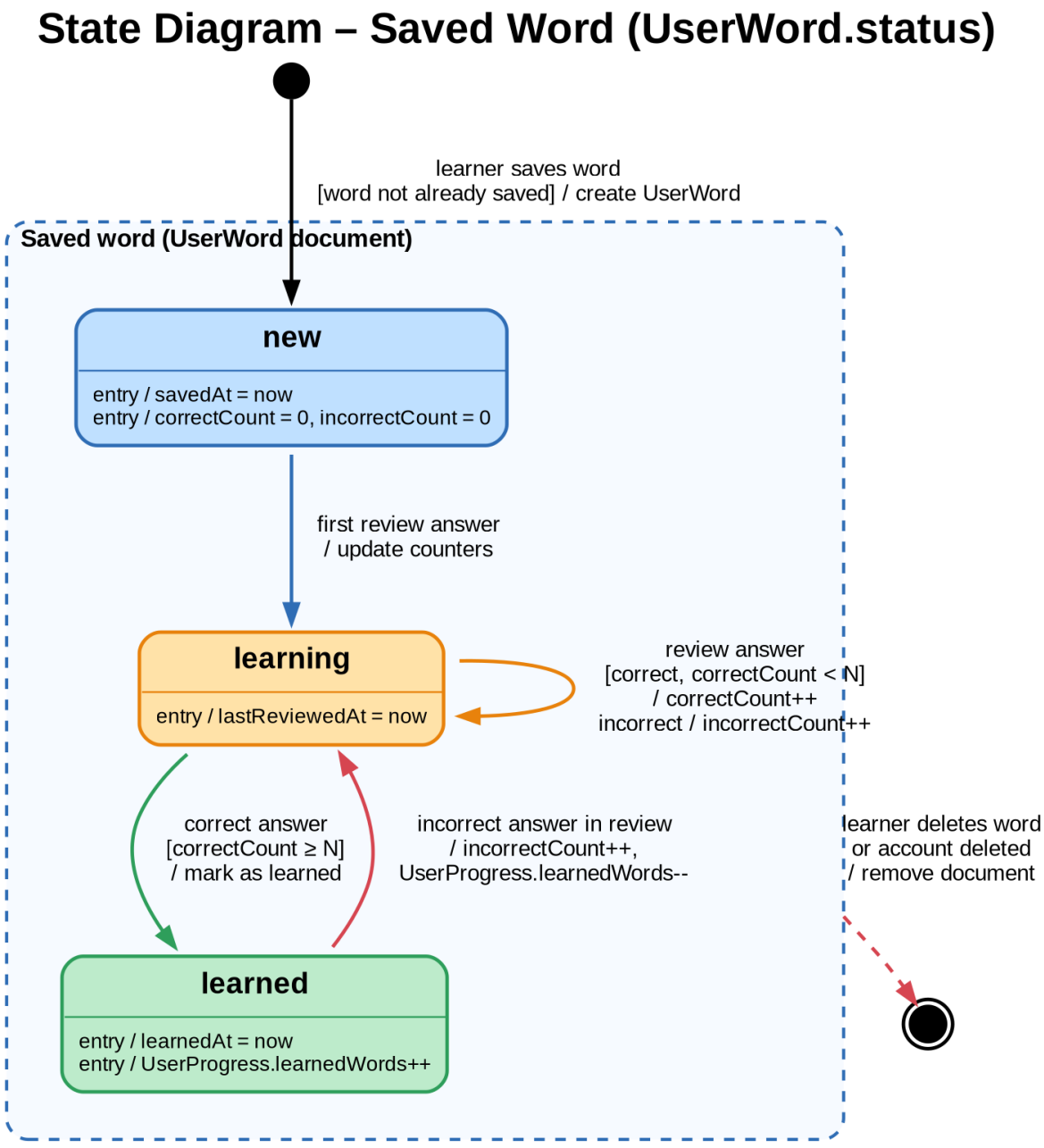


Figure 1: State diagram of a saved word (UserWord.status)

#	From	Event	Guard	Action	To
1	Initial	Learner saves a word	Word is not already saved	Create UserWord; savedAt = now; counters = 0	new
2	new	First review answer	–	Update correctCount or incorrectCount; lastReviewedAt = now	learning
3	learning	Review answer	Correct and correctCount < N, or incorrect	correctCount++ or incorrectCount++	learning
4	learning	Correct review answer	correctCount ≥ N	learnedAt = now; UserProgress.learnedWords++	learned
5	learned	Incorrect answer in review	–	incorrectCount++; UserProgress.learnedWords--	learning
6	Any saved state	Learner deletes the word, or the account is deleted	–	Remove the vocabulary document and its local copy	Final

Note: N is the number of correct answers required to mark a word as learned. It is a project decision and should be defined as a single constant shared by the Vocabulary ViewModel and the Database Design.

4.3 State Diagram – Lesson

A lesson is one AI-generated conversation session for a learner and a scenario (UC-04). It is created in the generated state, moves to in_progress when the learner submits the first response, stays there while the

practice continues (UC-05), and becomes completed when the learner finishes and the AI evaluation is stored. Completed lessons remain as conversation history.

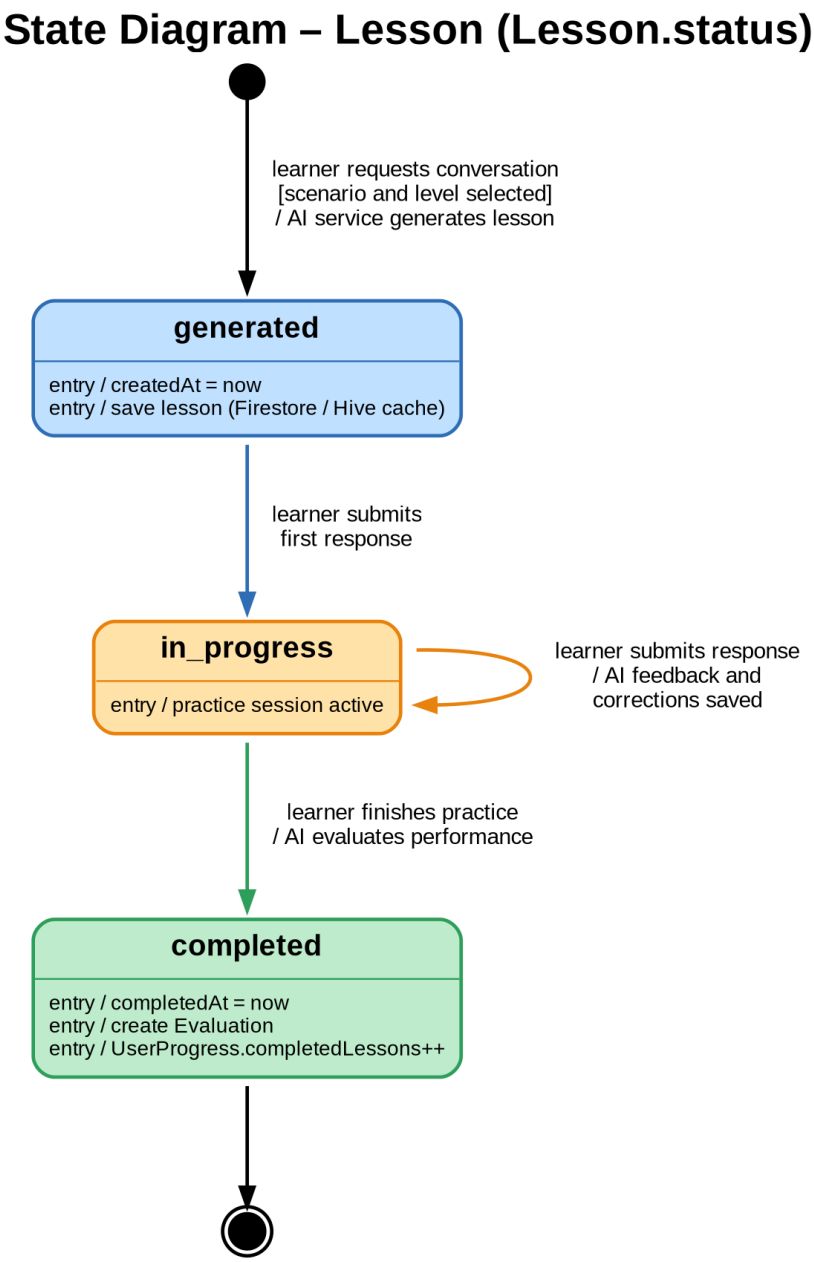


Figure 2: State diagram of a lesson (Lesson.status)

#	From	Event	Guard	Action	To
1	Initial	Learner requests a new conversation	Scenario and level are selected	AI service generates the lesson; save it (Firestore, optional Hive cache); createdAt = now	generated
2	generated	Learner submits the first response	–	Store the message; AI feedback; create Correction records for detected mistakes	in_progress
3	in_progress	Learner submits another response	–	Store the message and corrections	in_progress
4	in_progress	Learner finishes the practice	–	AI evaluates; create Evaluation; completedAt = now; UserProgress.completedLessons++	completed
5	completed	–	–	Lesson is kept as conversation history	Final

Error handling: if the AI service fails during transitions 1, 2, 3 or 4, the lesson stays in its current state, an error message is displayed, and the learner can retry (alternative flows of UC-04 and UC-05).

4.4 State Diagram – Level Assessment

A level assessment records how the learner's English level was determined (UC-02). A placement test takes time, so the record starts in in_progress and may be resumed later. A level that is chosen manually, or updated automatically from the learner's performance, is stored directly as completed.

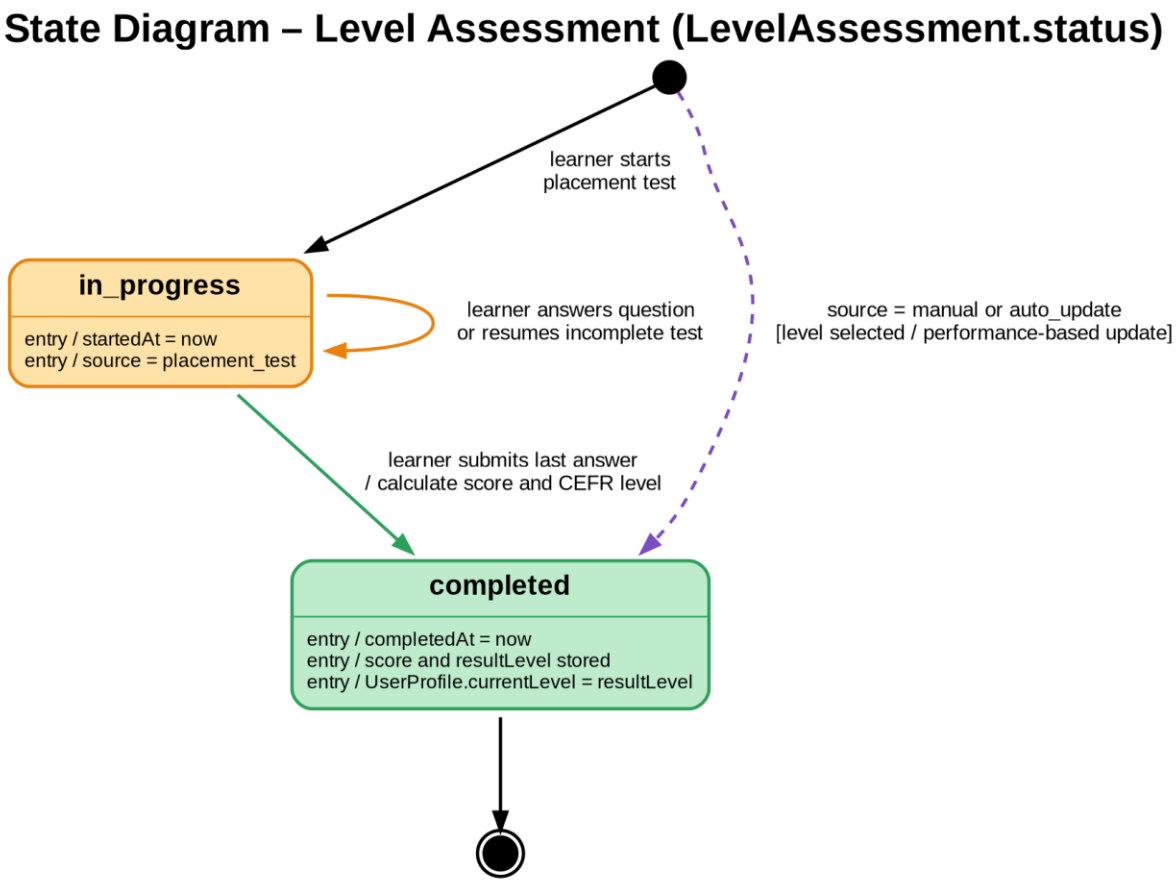


Figure 3: State diagram of a level assessment (LevelAssessment.status)

#	From	Event	Guard	Action	To
1	Initial	Learner starts the placement test	–	Create LevelAssessment; source = placement_test; startedAt = now	in_progress
2	in_progress	Learner answers a question or returns to an unfinished test	Test is incomplete	Save the answer; ask the learner to complete the assessment	in_progress
3	in_progress	Learner submits the last answer	All questions answered	Calculate score and CEFR level; completedAt = now; UserProfile.currentLevel = resultLevel	completed
4	Initial	Learner selects a level, or the system updates the level from performance	source = manual or auto_update	Create a completed record; update UserProfile.currentLevel	completed
5	completed	–	–	Record is kept as level history	Final

4.5 State Diagram – Sync Operation

Data changed while the device is offline, or when cloud synchronization is not available, is stored locally and queued as a SyncOperation in the Hive syncQueueBox (UC-09). The operation stays pending until connectivity returns, is sent to Cloud Firestore while syncing, and is removed from the queue when the write succeeds. A failed write returns it to pending and increases retryCount, so no learner data is lost.

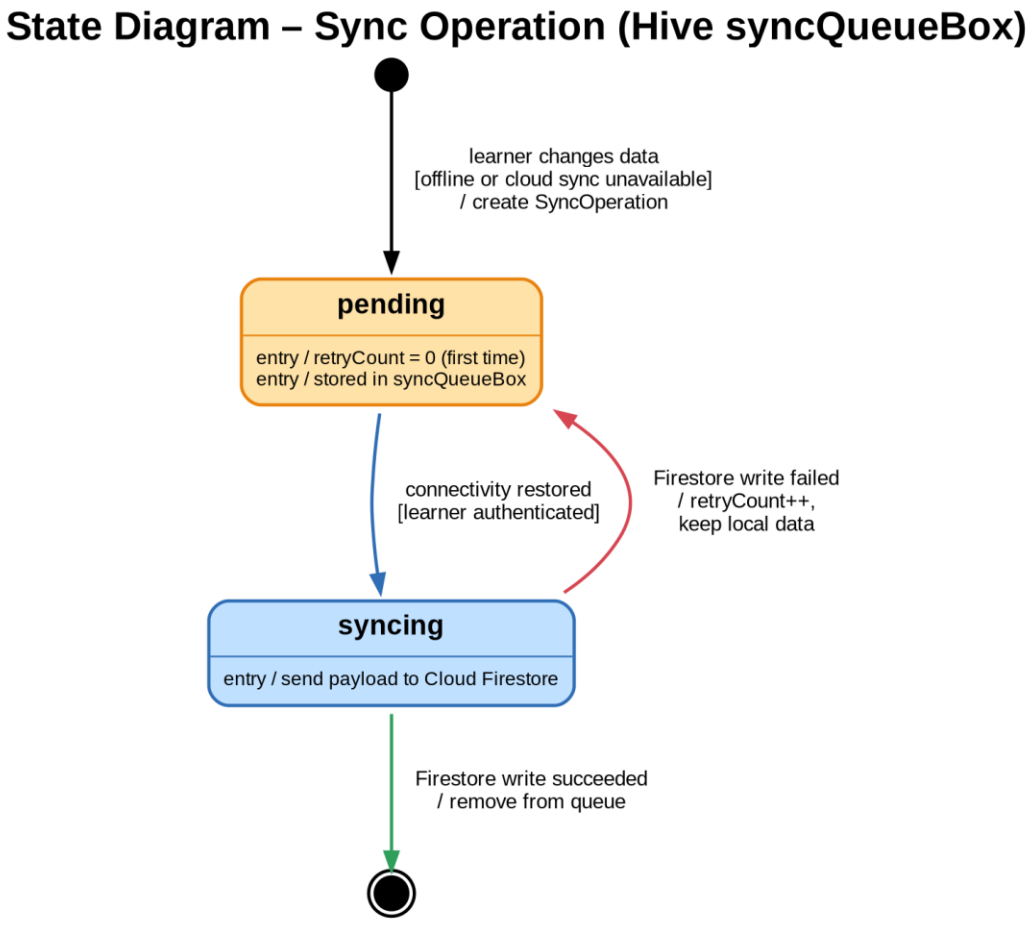


Figure 4: State diagram of a sync operation (Hive syncQueueBox)

#	From	Event	Guard	Action	To
1	Initial	Learner changes data (save word, update progress)	Device is offline, or cloud sync is unavailable	Create SyncOperation (retryCount = 0) in syncQueueBox; keep the data locally	pending
2	pending	Connectivity restored	Learner is authenticated	Send the payload to Cloud Firestore	syncing
3	syncing	Firestore write succeeded	–	Remove the operation from the queue; update lastSyncAt	Final
4	syncing	Firestore write failed	–	retryCount++; keep local data; try again later	pending

Note: the schema stores a SyncOperation with an operation type and a retryCount but no status field, so pending and syncing describe the life cycle of the queue entry rather than a stored attribute.

5. Class Diagram

5.1 Overview

The class diagram shows the static structure of AI-Lingo: its classes, their attributes and methods, and the relationships between them. It is built from two sources that were already defined in this document:

- **Domain model** – derived from the logical schema and ER diagram (Database Design). Each entity becomes a class, relationship cardinalities become multiplicities, and attribute names follow the Firestore field names (camelCase).
- **Application classes** – derived from the MVVM architecture (Software Architecture). The View, ViewModel and Model layers are expanded into screens, ViewModels, repositories and services, matching the processes P1–P6 of the Data Flow Diagram and the sequence diagrams.

To keep every diagram readable, the structure is presented in several figures. Names are identical across all figures and across the ER, DFD and sequence diagrams.

Notation key

Element	Meaning
+ / -	Public member / private member. Attributes are private; methods are public.
Type?	Nullable attribute (for example, completedAt is null until the lesson is completed).
Solid line with multiplicity	Association between two classes, for example 1 and 0..* (one to many).
Line with filled diamond	Composition: the part cannot exist without the whole (for example, a Message belongs to one Lesson).
Line with hollow diamond	Aggregation: the whole holds a collection of parts.
Dashed arrow	Dependency: one class uses another.
<<association class>>	A class that carries attributes of a many-to-many relationship (LessonWord, UserWord).
<<enumeration>>	A fixed set of allowed values.

Colour	Meaning
Blue	User module (domain figures); ViewModel layer (architecture figures).
Purple	Conversation module: categories, scenarios, lessons, messages.
Green	Vocabulary module: words, translations, saved words.
Orange	Assessment and feedback: level assessment, corrections, evaluation; also Hive models.
Rose / Teal / Amber / Grey	Architecture figures: View / Repository / Service / External systems.

5.2 Domain Model

The domain model is split into three figures by module. Classes that appear only as a reference in another figure are drawn with their name only.

5.2.1 User module

UserProfile holds the learner's profile, language settings and current CEFR level. Each learner has exactly one UserProgress summary (composition), any number of LevelAssessment records, and a list of interests selected from the shared Category list.

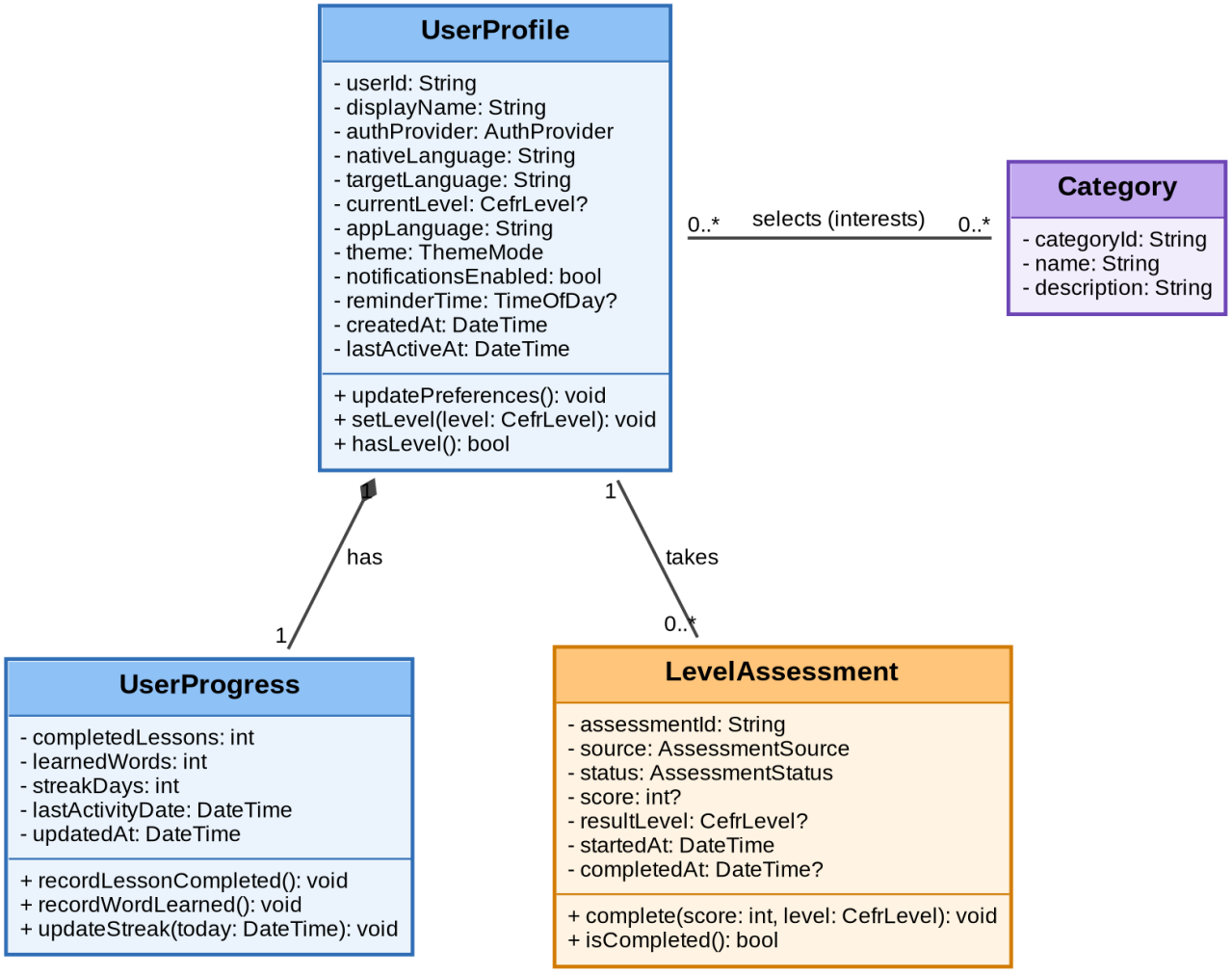


Figure 5: Class diagram – user module

5.2.2 Conversation module

A Scenario belongs to an optional Category and is either predefined (createdBy is empty) or created by a learner. A learner practises a scenario through a Lesson, which contains Messages and, once completed, one Evaluation. Corrections are attached to the learner's messages.

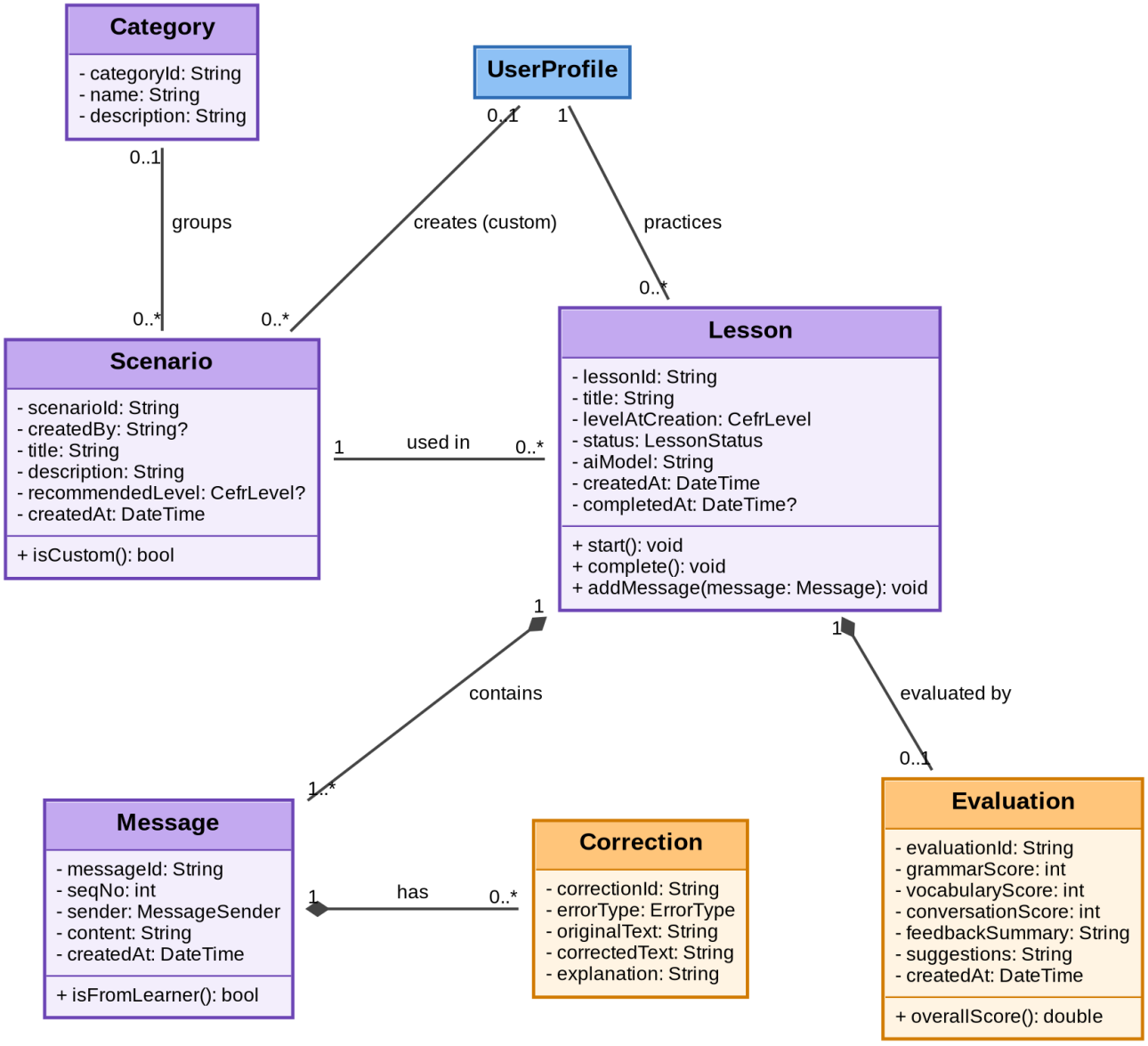


Figure 6: Class diagram – conversation module

5.2.3 Vocabulary module

Word is the shared dictionary entry with its translations. LessonWord records which words a lesson introduced and whether they were new, and UserWord records the words a learner saved together with the review history that decides when a word is learned.

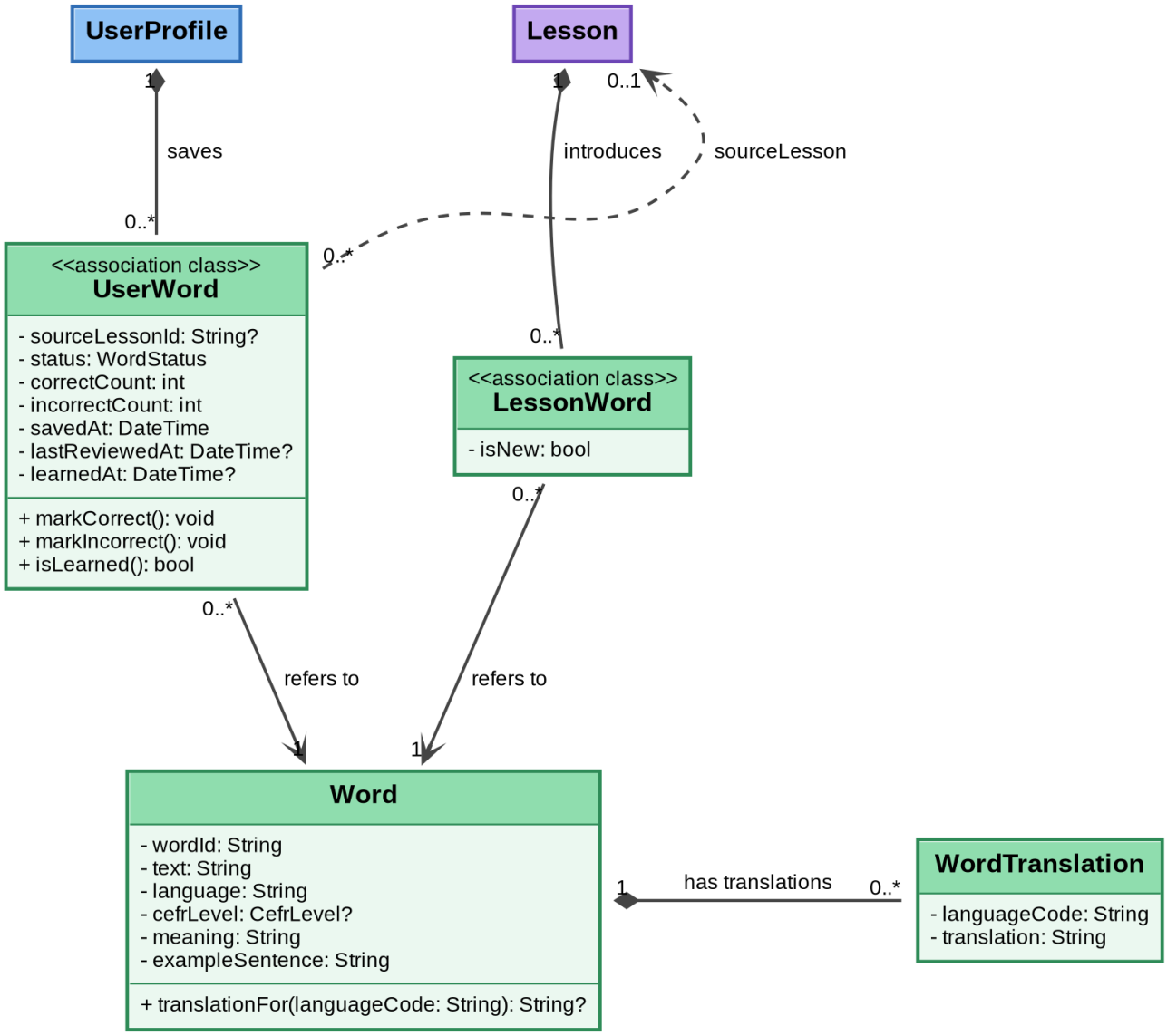


Figure 7: Class diagram – vocabulary module

5.2.4 Relationships

Class A	Relationship	Class B	Multiplicity (A : B)	Type	ER source
UserProfile	has	UserProgress	1 : 1	Composition	USER – USER_PROGRESS
UserProfile	takes	LevelAssessment	1 : 0..*	Association	USER – LEVEL_ASSESSMENT
UserProfile	selects (interests)	Category	0..* : 0..*	Association	USER – CATEGORY (USER_INTEREST)
Category	groups	Scenario	0..1 : 0..*	Association	CATEGORY – SCENARIO
UserProfile	creates (custom)	Scenario	0..1 : 0..*	Association	USER – SCENARIO
UserProfile	practices	Lesson	1 : 0..*	Association	USER – LESSON
Scenario	used in	Lesson	1 : 0..*	Association	SCENARIO – LESSON
Lesson	contains	Message	1 : 1..*	Composition	LESSON – MESSAGE
Message	has	Correction	1 : 0..*	Composition	MESSAGE – CORRECTION
Lesson	evaluated by	Evaluation	1 : 0..1	Composition	LESSON – EVALUATION
Lesson	introduces	Word (via LessonWord)	1 : 0..*	Composition + reference	LESSON – WORD (LESSON_WORD)
UserProfile	saves	Word (via UserWord)	1 : 0..*	Composition + reference	USER – WORD (USER_WORD)
Word	has translations	WordTranslation	1 : 0..*	Composition	WORD – WORD_TRANSLATION
UserWord	sourceLesson	Lesson	0..* : 0..1	Dependency	USER_WORD.source_lesson_id

Note: the ER diagram shows the Lesson – Evaluation relationship as 1 : 1. In the class diagram it is 0..1 because an evaluation is created only when the lesson is completed (see the Lesson state diagram). Derived values that the schema deliberately does not store are provided as methods: Scenario.isCustom() (createdBy is not null) and Evaluation.overallScore() (average of the three scores).

5.3 Enumerations

The enumerations below define the allowed values of the status and type attributes. Their stored values in Firestore and Hive are the lowercase snake_case values defined in the schema (for example, in_progress, placement_test, auto_update); newWord is stored as new because new is a reserved word in Dart.

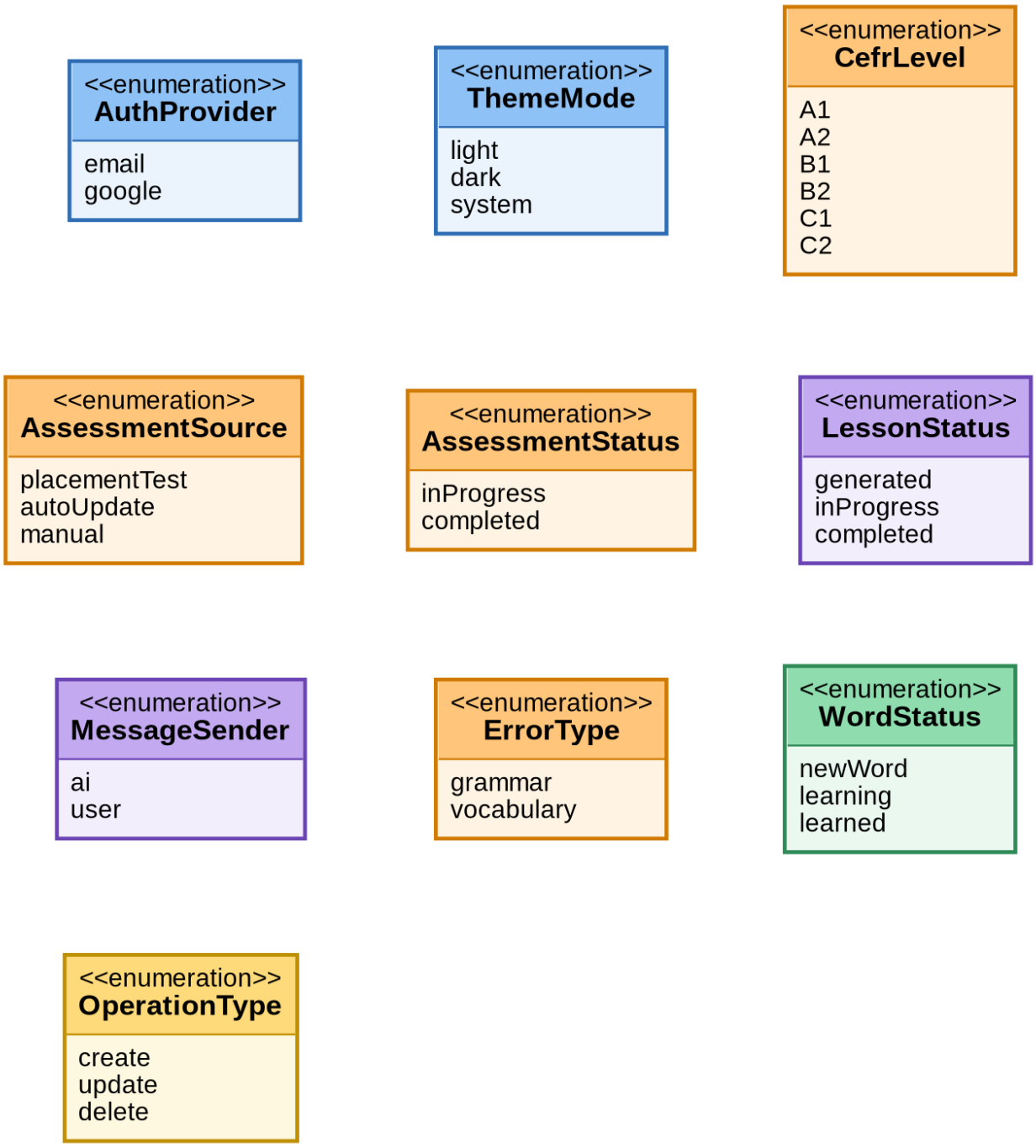


Figure 8: Enumerations used by the domain classes

5.4 Application Architecture (MVVM)

The figure below shows how the Model–View–ViewModel layers are realised as classes. Dependencies point in one direction only: a View observes one ViewModel, a ViewModel uses repositories, a repository uses services, and a service communicates with one external system or storage. Domain classes from Section 5.2 are the data passed between the layers.

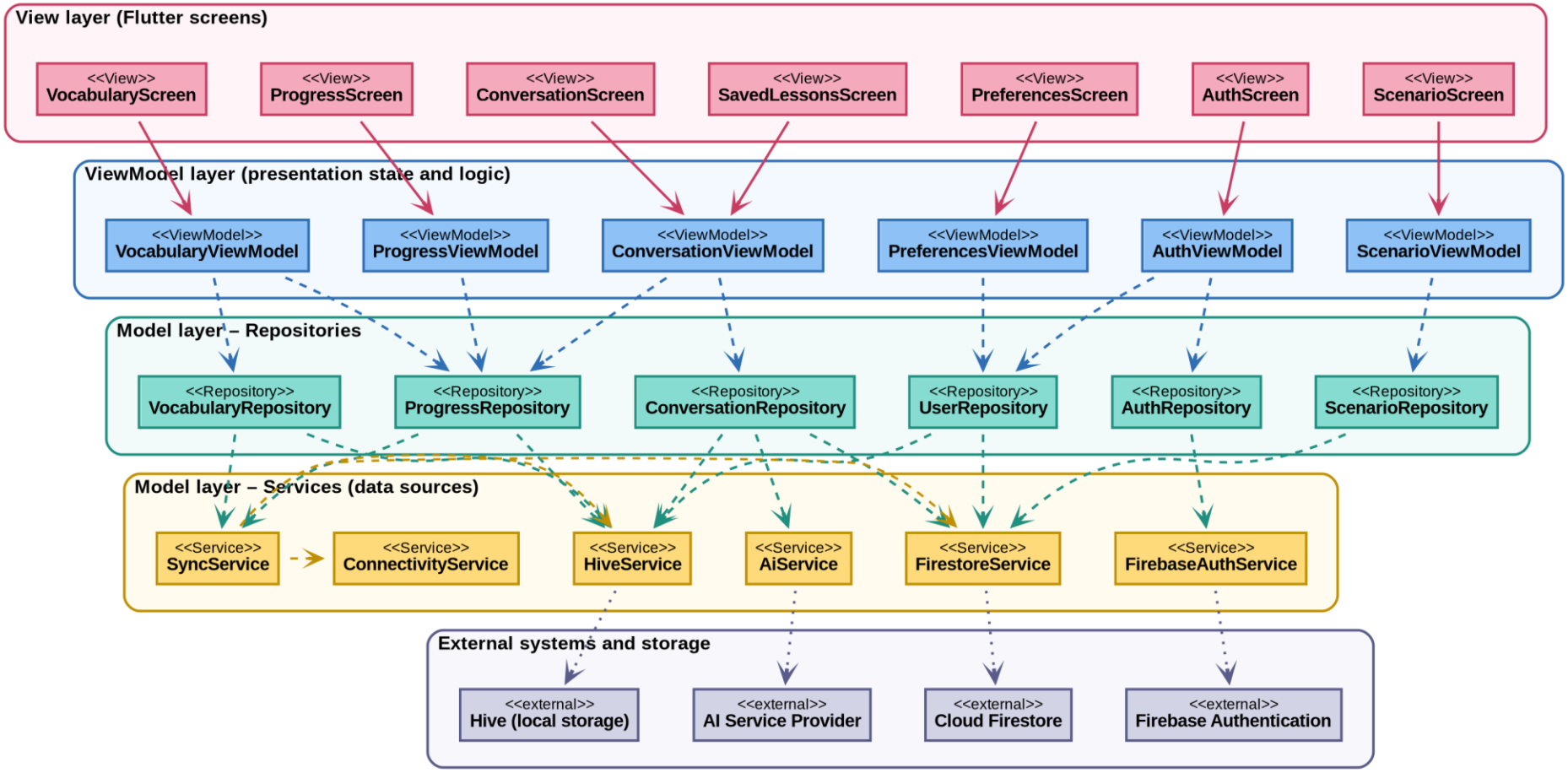


Figure 9: Class diagram – MVVM layers and dependencies

Dependency rules

- Views never call repositories or services; they display ViewModel state and forward user actions.
- ViewModels never access Firebase, Hive or the AI service directly; they call repositories only.
- Repositories decide whether data comes from Cloud Firestore, Hive or the AI service, and create a SyncOperation when cloud synchronization cannot be completed.
- Only the services know the external APIs, so a provider (for example the AI service) can be replaced without changing the layers above it.

5.5 View Layer

The View layer consists of the Flutter screens. Each screen is bound to one ViewModel and contains no business logic.

View (screen)	ViewModel	Use cases	Purpose
AuthScreen	AuthViewModel	UC-01	Registration, login and logout
PreferencesScreen	PreferencesViewModel	UC-02	Learning preferences, interests and English level
ScenarioScreen	ScenarioViewModel	UC-03	Browse predefined scenarios or create a custom one
ConversationScreen	ConversationViewModel	UC-04, UC-05	Generated conversation, practice, feedback
SavedLessonsScreen	ConversationViewModel	UC-08	Open saved lessons without internet connection
VocabularyScreen	VocabularyViewModel	UC-06	Saved words, review mode
ProgressScreen	ProgressViewModel	UC-07	Completed lessons, learned words, level

5.6 ViewModel Layer

ViewModels hold the presentation state (data, loading flag, error message), validate user input, and call the repositories. They correspond to the processes P1–P6 of the Data Flow Diagram.

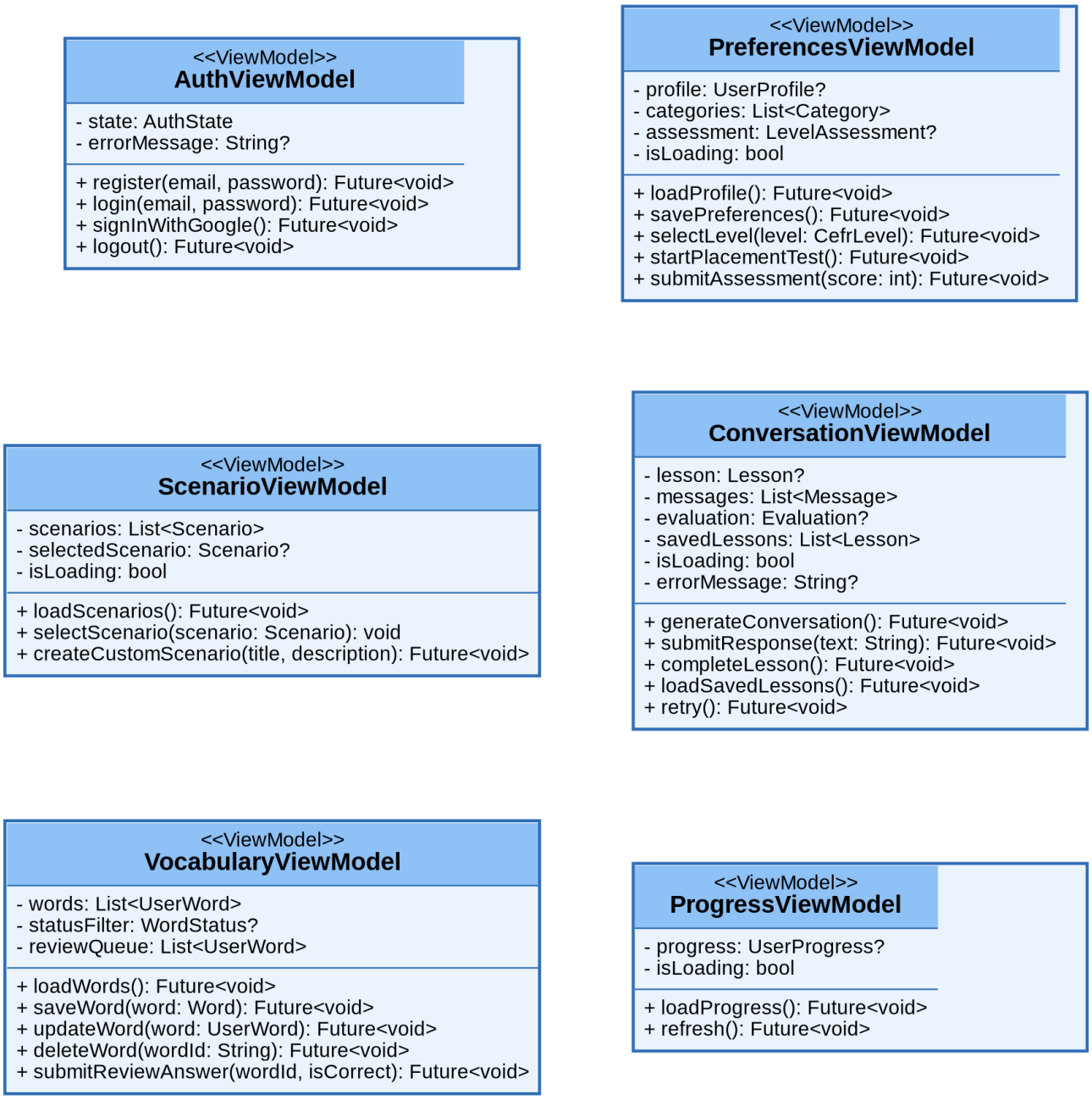


Figure 10: Class diagram – ViewModel layer

ViewModel	Responsibility	Repositories used
AuthViewModel	Registration, login, Google sign-in and logout; exposes the authentication state (P1).	AuthRepository, UserRepository
PreferencesViewModel	Loads and saves language settings, interests and level; runs the placement test (P2).	UserRepository
ScenarioViewModel	Lists predefined and custom scenarios and keeps the selected scenario (P3).	ScenarioRepository
ConversationViewModel	Generates the conversation, submits responses, shows feedback, completes the lesson and loads saved lessons (P3, P4).	ConversationRepository, ProgressRepository
VocabularyViewModel	Adds, edits, deletes and reviews words; updates the word status (P5).	VocabularyRepository, ProgressRepository
ProgressViewModel	Loads the progress summary for the dashboard (P6).	ProgressRepository

5.7 Repository Layer

Repositories hide where data is stored. They convert between domain classes and Firestore documents or Hive models, choose the data source, and pass requests that need AI to the AI service.

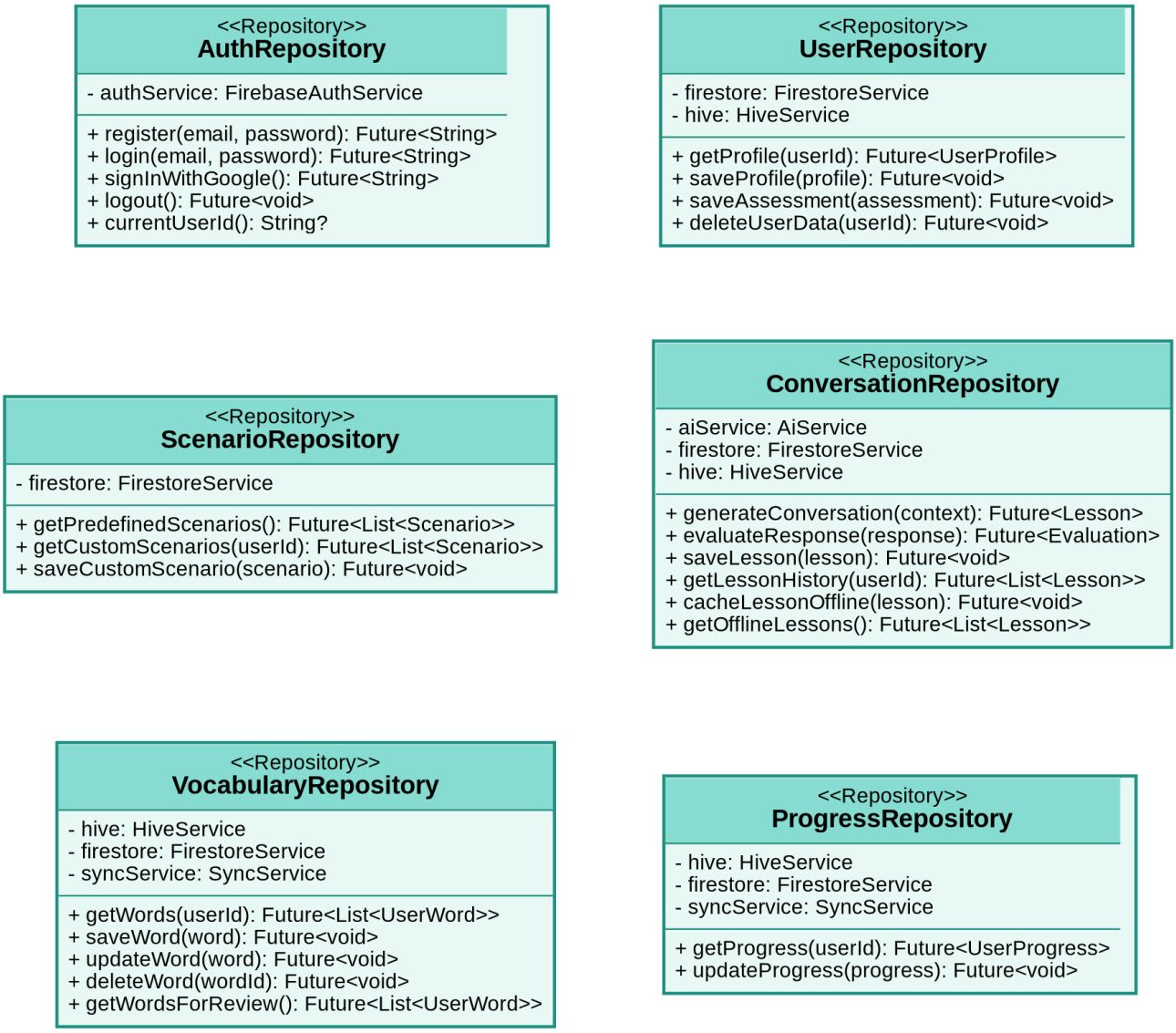


Figure 11: Class diagram – repository layer

Repository	Responsibility	Services used
AuthRepository	Authenticates the learner and returns the user id.	FirebaseAuthService
UserRepository	Stores the profile, interests and level assessments; deletes all user data when the account is deleted.	FirestoreService, HiveService
ScenarioRepository	Reads predefined scenarios and stores custom scenarios.	FirestoreService
ConversationRepository	Builds the prompt, requests generation and evaluation, saves lessons and messages, and caches lessons for offline use.	AiService, FirestoreService, HiveService
VocabularyRepository	Reads and writes saved words locally first and queues changes for cloud synchronization.	HiveService, FirestoreService, SyncService
ProgressRepository	Reads and updates the progress counters used by the dashboard.	HiveService, FirestoreService, SyncService

5.8 Service Layer

Services wrap the external systems. SyncService keeps a queue of SyncOperation objects (the Hive syncQueueBox) and uses ConnectivityService to know when synchronization can run.

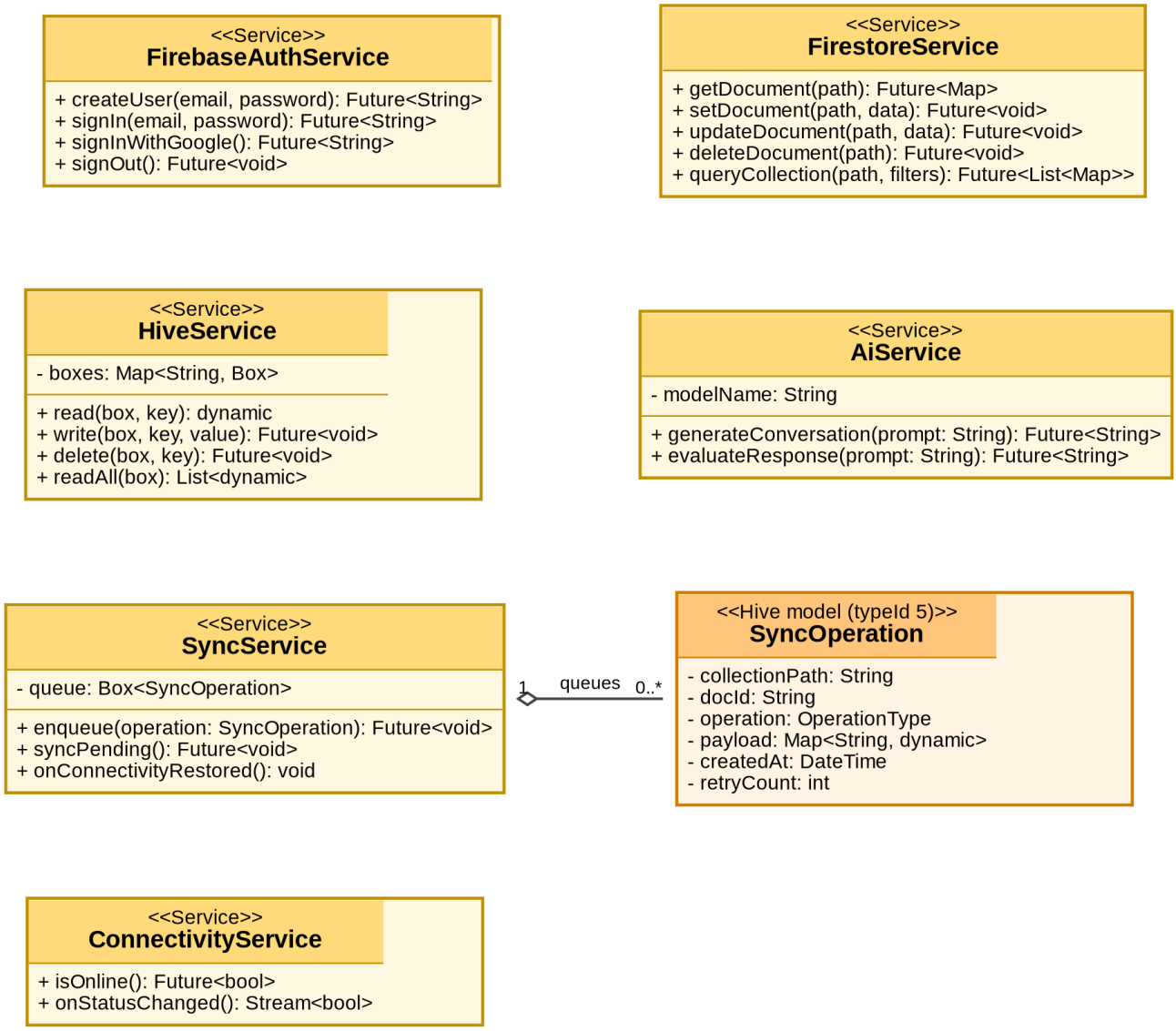


Figure 12: Class diagram – service layer

Service	Responsibility	External system / storage
FirebaseAuthService	Creates users, signs in (email/password, Google) and signs out.	Firebase Authentication
FirestoreService	Generic read, write, update, delete and query operations on documents.	Cloud Firestore
HiveService	Reads and writes the local boxes: vocabularyBox, offlineLessonsBox, progressBox, syncQueueBox, settingsBox.	Hive (local storage)
AiService	Sends prompts for conversation generation and evaluation; only the information needed is sent.	AI Service Provider
SyncService	Queues operations and synchronizes them when connectivity is restored.	Hive and Cloud Firestore (through their services)
ConnectivityService	Reports whether the device is online and notifies when the status changes.	Device network status

5.9 Traceability: Use Cases to Classes

The table links every use case to the classes that implement it, so that each requirement can be traced from the use case to the design.

Use case	View	ViewModel	Repository	Services
UC-01 Register / Login	AuthScreen	AuthViewModel	AuthRepository, UserRepository	FirebaseAuthService, FirestoreService
UC-02 Select English Level	PreferencesScreen	PreferencesViewModel	UserRepository	FirestoreService, HiveService
UC-03 Choose Conversation Scenario	ScenarioScreen	ScenarioViewModel	ScenarioRepository	FirestoreService
UC-04 Generate AI Conversation	ConversationScreen	ConversationViewModel	ConversationRepository	AiService, FirestoreService, HiveService
UC-05 Evaluate Performance and Receive Feedback	ConversationScreen	ConversationViewModel	ConversationRepository	AiService, FirestoreService
UC-06 Manage Vocabulary	VocabularyScreen	VocabularyViewModel	VocabularyRepository	HiveService, SyncService
UC-07 Track Learning Progress	ProgressScreen	ProgressViewModel	ProgressRepository	HiveService, FirestoreService
UC-08 Access Saved Lessons Offline	SavedLessonsScreen	ConversationViewModel	ConversationRepository	HiveService
UC-09 Store and Synchronize Learning Data	–	VocabularyViewModel, ProgressViewModel	VocabularyRepository, ProgressRepository	SyncService, ConnectivityService, FirestoreService, HiveService